Kincskereső robot

Csapatnév: Keri1

Csapattagok: Horváth Dávid, Szabó Zsombor, Varga Bence

1. Az alkalmazás rövid leírása

Az alkalmazás egy böngészőben futó webes játék, amelyben a felhasználó MineScript nyelven írt programmal irányít egy kincskereső robotot egy labirintusszerű bányában. A program szerkeszthető, fájlból importálható és fájlba exportálható, futtatás előtt szintaktikailag ellenőrizhető, futás közben pedig lassítva látható a robot mozgása, az éppen végrehajtott sor, a felfedezett terület, a kibányászott arany és a lépésszám. A felhasználók saját térképeket készíthetnek, az eredmények térképenként ranglistára kerülnek.

Felhasznált technológiák

- **Backend:** Node.js, Express (REST API és a statikus fájlok kiszolgálása), mysql2 (adatbázis-kapcsolat).
- **Adatbázis:** MySQL (vagy MariaDB).
- **Frontend:** sima HTML, CSS és JavaScript, keretrendszer nélkül.

Telepítés és indítás

- Szükséges: Node.js (18 vagy újabb) és egy futó MySQL/MariaDB szerver (pl. XAMPP).
- A projekt mappájában: npm install, majd npm start.
- Böngészőben: http://localhost:3000
- Az adatbázis-kapcsolat adatai a projekt gyökerében lévő .env fájlban állíthatók (DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME, PORT); az npm start ezt automatikusan betölti. Alapértelmezés: localhost, 3306, root, üres jelszó, robotszimulator adatbázis, 3000-es port.
- Linux szerverre (Apache + MySQL) történő telepítés lépései a TELEPITES.md fájlban találhatók: az alkalmazás systemd szolgáltatásként fut, az Apache reverse proxyként továbbítja a kéréseket.
- Indításkor a szerver automatikusan létrehozza az adatbázist és a táblákat (users, maps, results), és ha még nincs felhasználó, feltölti egy bemutató játékkörnyezettel.

Bemutató játékkörnyezet

- Felhasználók: demo / demo és bela / bela.
- Térképek: „Bevezető bánya” (a feladatleírás példája), „Folyosó”, „Labirintus”.
- Néhány előre rögzített eredmény a „Folyosó” térképen, így a ranglista rögtön kipróbálható.
- A peldak mappában egy importálható térképfájl (terkep.txt) és programfájl (program.txt) található.

2. A funkciók elérése

Bejelentkezés és felhasználókezelés

A bejelentkezés és a regisztráció külön oldalon (login.html) található. Az oldal tetején lévő kapcsolóval lehet váltani a Bejelentkezés és a Regisztráció űrlap között. Regisztráció szabadon lehetséges (felhasználónév 3–20 karakter, jelszó legalább 4 karakter, a jelszót kétszer kell megadni). Sikeres regisztráció után az oldal automatikusan a bejelentkezésre vált, kitöltött felhasználónévvel. Belépés után az alkalmazás főoldalára kerül a felhasználó, ahol a fejlécben látszik a neve és a Kijelentkezés gomb. Bejelentkezés nélkül a főoldal megnyitása visszairányít a bejelentkezési oldalra; a szerver minden kérésnél ellenőrzi a munkamenet-azonosítót.

Játék fül – programszerkesztés és futtatás

- **Térkép választása:** a legördülő listából választható a térkép, alatta látszik a térkép, a robot és a statisztika (arany, lépés, aktuális sor). A térkép mérete a rendelkezésre álló helyhez igazodik, így a legnagyobb, 15×15-ös térkép is görgetés nélkül, egészben látszik.
- **Kódszerkesztő:** a bal oldali szövegmezőben írható a program, a jobb oldali, sorszámozott nézet mutatja a sorokat, kiemelve a futó sort (sárga) vagy a hibás sort (piros).
- **Importálás / exportálás:** az Importálás gombbal .txt programfájl tölthető be. Ha a fájl szintaktikailag hibás, az alkalmazás hibaüzenetet ad a sor számával, és nem tölti be. Az Exportálás gomb a program.txt fájlba menti a programot.
- **Ellenőrzés:** az Ellenőrzés gomb lefuttatja a szintaxis-ellenőrzést, és kiírja a hibát a sor számával.
- **Futtatás:** a Futtatás gomb ellenőrzi a programot, alaphelyzetbe állítja a térképet és a robotot, majd lassítva végrehajtja. A sebesség (lassú, közepes, gyors) futás közben is állítható. A print kimenete a Konzol részen jelenik meg. A Leállítás gombbal a futás megszakítható.
- **Felfedezett terület:** a felfedezett mezők (ahol a robot járt, illetve amelyekre isThere hívás történt) normál színűek, a még felfedezetlenek sötétítve látszanak.
- **Hibák:** futásidejű hiba vagy a lépéslimit túllépése esetén a felület kiírja a hibakódot (pl. E_BEDROCK_HIT, INSTRUCTION_COUNT_LIMIT_EXCEEDED), a hibás sor számát és egy magyar nyelvű magyarázatot, a hibás sort pedig pirossal jelöli.
- **Eredmény mentése:** a futás végén (sikeres futás vagy hiba esetén) az eredmény automatikusan a ranglistára kerül.

Térképszerkesztő fül

- A legördülő listában „Új térkép” vagy egy meglévő térkép választható.
- A Sorok és Oszlopok mezőkkel (1–15) és a Méret alkalmazása gombbal állítható a méret; a meglévő mezők megmaradnak.
- Az „Elhelyezendő elem” csoportban kiválasztható az üres kamra, az arany, a fal vagy a robot a négy kezdőirány egyikével, majd a térkép mezőire kattintva (vagy Tab + Enter/Szóköz billentyűkkel) elhelyezhető. A szerkesztőben is a teljes térkép látszik görgetés nélkül. Új robot lerakásakor a régi automatikusan eltűnik, így mindig legfeljebb egy robot van.
- Mentés: a saját térkép felülírása; Mentés újként: új térkép létrehozása. Mentés előtt a kliens és a szerver is ellenőrzi a szabályokat (méret, pontosan egy robot, csak érvényes jelek).
- Importálás / Exportálás: a feladatleírás szerinti formátumú .txt fájl. Hibás fájl esetén érthető hibaüzenet jelenik meg (pl. melyik sor hossza rossz, hány robot van), és a tartalom nem töltődik be.
- Törlés: csak a saját térkép törölhető, megerősítés után.

A térképek minden felhasználó számára közösek (mindenki játszhat rajtuk), de módosítani és törölni csak a létrehozó tudja. Más térképe „Mentés újként” lemásolható.

Ranglista fül

A térkép kiválasztása után táblázat mutatja az eredményeket: előrébb az áll, aki több aranyat gyűjtött, azonos arany esetén akinek kevesebb lépés kellett. Minden felhasználó a legjobb eredményével szerepel. Egy térkép módosításakor a hozzá tartozó eredmények törlődnek, mert a régi eredmények már nem összehasonlíthatók.

3. Elkészült és ki nem készült részek

A feladatleírás minden alapkövetelménye elkészült: bejelentkezés, térképszerkesztő, térkép és program import/export, szintaxis-ellenőrzés a hibás sor jelzésével, lassított futtatás, felfedezett terület, konzol, mind a hét futásidejű hibakód és az 1000 soros limit, valamint a ranglista.

Egyszerűsítések, ismert korlátok:

- A munkamenetek a szerver memóriájában tárolódnak, ezért a szerver újraindítása után újra be kell jelentkezni.
- A programot a böngésző hajtja végre, az eredményt a kliens küldi a szervernek, így a szerver nem ellenőrzi újra a futtatás eredményét.
- A step forward n utasítás egyetlen lépésként animálódik (a robot a sor végrehajtása után jelenik meg az új helyén).
- Nincs külön adminisztrátori felület és jelszó-visszaállítás.
- A kódszerkesztő nem színezi a szintaxist.

4. Plusz szolgáltatások

- Szabad regisztráció jelszó-megerősítéssel, külön bejelentkezési oldal; a jelszavak sózott scrypt hash-sel tárolva.
- Modern, sötét témájú felület: egységes színvilág (arany kiemelőszín a bányás témához), lekerekített kártyák, jól elkülönülő mezőszínek a térképen, kiemelt aktuális és hibás programsor.
- Állítható futási sebesség, futás közben is.
- Külön Ellenőrzés gomb a futtatás nélküli szintaxis-ellenőrzéshez, magyar nyelvű, konkrét hibaüzenetekkel.
- Importált programfájl ellenőrzése betöltés előtt.
- A térkép érvényességét a szerver is ellenőrzi, így hibás térkép közvetlen API-hívással sem kerülhet az adatbázisba.
- Közös térképek tulajdonosi jogosultsággal, más térképeinek lemásolása.
- A ranglista felhasználónként a legjobb eredményt mutatja.
- Automatikus adatbázis-létrehozás és bemutató adatok első indításkor.
- Akadálymentesség: minden mezőnek van címkéje, a térképszerkesztő billentyűzettel is kezelhető, a mezők felolvasható leírást kapnak (pl. „2. sor, 3. oszlop: arany”), a hibaüzenetek és a konzol aria-live területen jelennek meg, jól látható fókuszkeret, megfelelő színkontraszt a sötét témában is, mobilon egymás alá rendeződő elrendezés.

5. Felmerült problémák és megoldásaik

- **Behúzás alapú blokkok:** a MineScript blokkjait a behúzás jelöli. Megoldás: a sorokat először tokenekre bontjuk és eltároljuk a behúzás mértékét, majd egy rekurzív blokkolvasó függvény építi fel az utasításfát: a blokk az első sora behúzásával azonos sorokból áll, a nagyobb

behúzás új blokkot nyit, a kisebb lezárja. Így a blokkonként eltérő, de blokkon belül egységes behúzás is helyesen kezelhető.

- **Lassított, megállítható futtatás:** a programnak lassítva kell futnia, és minden sor után frissülnie kell a felületnek. Megoldás: az értelmező JavaScript generátorfüggvény (function*), amely minden feldolgozott sor után átadja a vezérlést (yield). A felület így lépésenként, várakozással hajtja végre, és a leállítás is egyszerű.
- **Lépésszámolás:** a while sora minden feltétel-kiértékelésnél számít, az else sora csak akkor, ha az else ág lefut. Megoldás: a számlálás egy közös függvényben történik, amely a limit (1000) elérésekor INSTRUCTION_COUNT_LIMIT_EXCEEDED hibát dob, mielőtt újabb sort dolgozna fel.
- **Típusellenőrzés sorrendje:** a kifejezések balról jobbra, operandusonként ellenőrzendők. Megoldás: minden operandust beolvasás után azonnal ellenőrzünk az adott műveletre, az = és <> esetén pedig a típusegyezést a második operandus után vizsgáljuk.
- **Közös ellenőrző kód:** a térképet a kliensnek (szerkesztő, import) és a szervernek (mentés) is ellenőriznie kell. Megoldás: a map.js fájl a böngészőben és a Node.js-ben is betölthető, így ugyanaz a kód ellenőriz mindkét helyen.
- **Kulcsszavak:** a kulcsszavak kis- és nagybetűs változatai sem lehetnek azonosítók (pl. True, IsThere). Megoldás: az azonosítót kisbetűsítve hasonlítjuk össze a kulcsszavak listájával.
- **Nagy térképek megjelenítése:** a nagyobb (pl. 15 oszlopos) térképek fix méretű mezőkkel nem fértek ki a kártyában, ezért görgetősávval jelentek meg, így a robot mozgása nem volt egyben követhető. Megoldás: a térkép oszlopai rugalmas méretűek (legfeljebb 2,2rem), a mezők pedig mindig négyzet alakúak maradnak, így a térkép a rendelkezésre álló helyhez zsugorodik.
- **Rejtett elemek megjelenése:** a bejelentkezési oldalon a kapcsoló ellenére mindkét űrlap látszott, mert az űrlapokra adott CSS megjelenítési szabály felülírta a böngésző hidden attribútumát. Megoldás: egy általános [hidden] { display: none !important; } szabály, amely minden elrejtett elemre érvényes.

6. Továbbfejlesztési ötletek

- Mezőnkénti animáció a step forward utasításnál, illetve léptetés gombbal soronkénti végrehajtás (debugger).
- Szintaxis-kiemelés és automatikus behúzás a kódszerkesztőben.
- Változók aktuális értékének megjelenítése futás közben.
- Szerveroldali újrafuttatás a ranglista-eredmények hitelesítéséhez.
- Pályák nehézségi szintje, értékelése és kedvencnek jelölése; kihívások időkorláttal.
- Köd: csak a felfedezett mezők tartalma látszódjon, így a játékosnak valóban „vakon” kelljen programoznia.

7. A forráskód felépítése

Fájl	**Feladata**
server.js	Express szerver, REST API (regisztráció, bejelentkezés, térképek, eredmények, ranglista), adatbázis létrehozása és bemutató adatok.
public/login.html, public/login.js	Külön bejelentkezési és regisztrációs oldal, váltással a két űrlap között.
public/index.html	Az alkalmazás főoldala (Játék, Térképszerkesztő, Ranglista fülek).
public/style.css	A sötét témájú megjelenés, CSS változókkal.

public/app.js	A főoldal logikája: API-hívások, térkép kirajzolása, szerkesztő, futtatás, ranglista.
public/minescript.js	MineScript szintaxis-ellenőrző (parser) és értelmező.
public/map.js	Térképfájl ellenőrzése és előállítása (kliens és szerver közös kódja).
peldak/	Példa térkép- és programfájl importáláshoz.

REST API

Végpont	**Leírás**
POST /api/register	Regisztráció
POST /api/login, POST /api/logout	Be- és kijelentkezés
GET /api/maps	Térképek listája
POST /api/maps, PUT /api/maps/:id, DELETE /api/maps/:id	Térkép létrehozása, módosítása, törlése
POST /api/results	Futás eredményének rögzítése
GET /api/leaderboard/:mapId	Ranglista egy térképhez